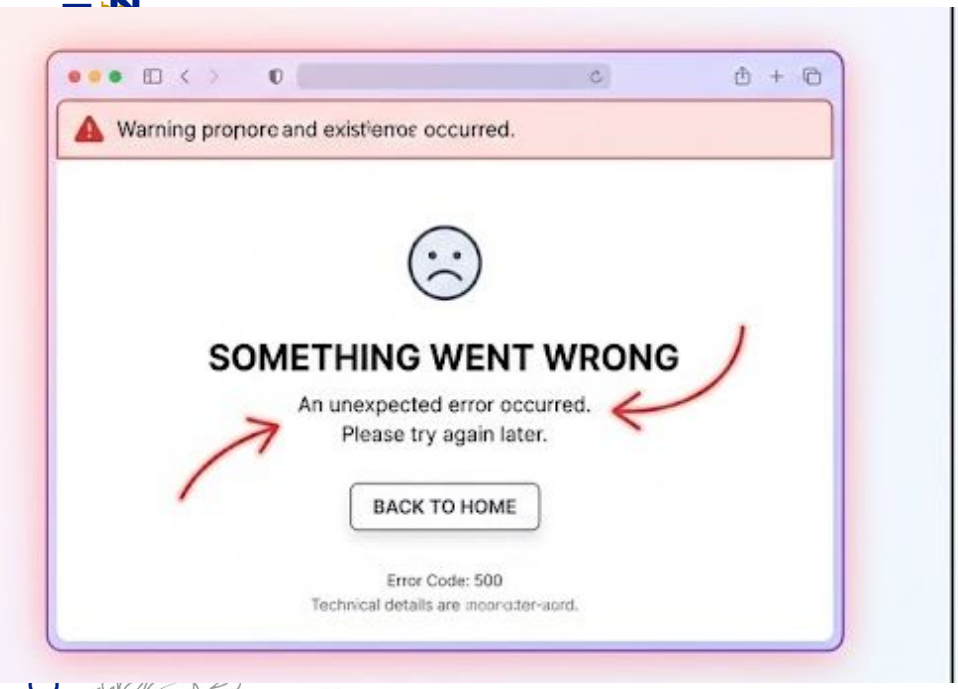
UNIVERSITY OF SZEGED
Department of Software Engineering
UNIVERSITAS SCIENTIARUM SZEGEDIENSIS

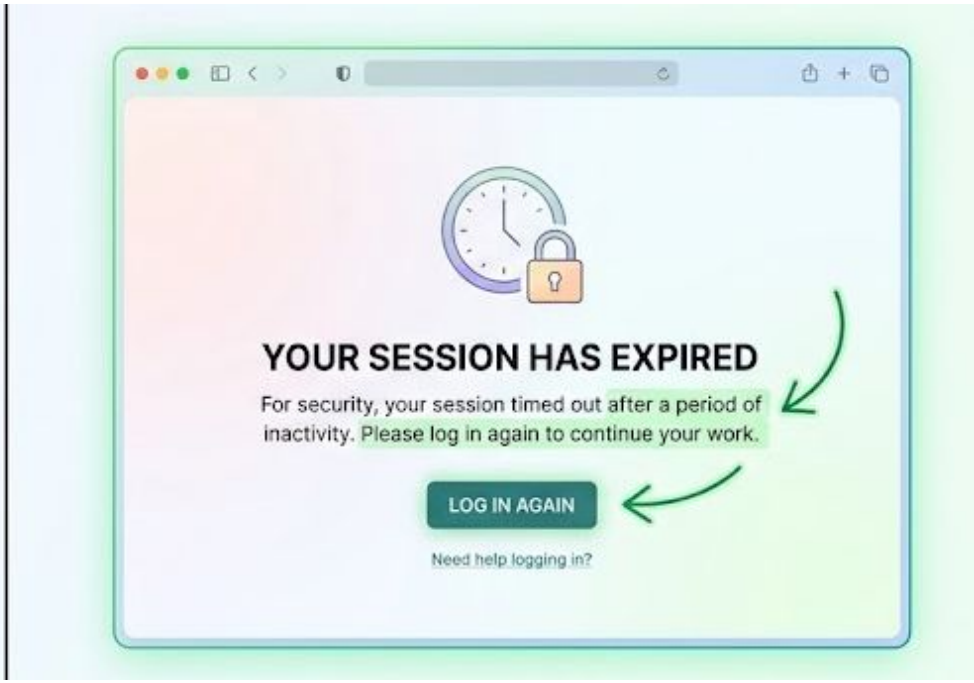
Software Architectures

Practice 7

Zoltán Richárd JÁNKI, PhD

Error messages





- Errors are not bugs to hide
- It is a communication channel between the system and the end user

Error handling as architectural decision

- Categorization
- Propagation
- Presentation

} architectural decision

- Potential problems
  - `catch (e) { console.log(e); }`
  - random HTTP status codes
  - raw database errors leaking to the frontend

UNIVERSITY OF SZEGED
Department of Software Engineering
UNIVERSITAS SCIENTIARUM SZEGEDIENSIS

Error Taxonomy

- deliberate, documented classification of every type of error your system can produce
- it affects every layer of your system
  - data layer reports constraint violations
  - business logic layer signals rule violations
  - API layer formats responses
  - frontend presents the responses
- Analogy
  - you would not store every piece of data as a string
  - **you should not treat every error as generic**

Find error origins by layer

- Data layer
  - Connection failures
  - Constraint violations
    - unique, foreign key, not null
  - Query timeouts
- Business logic layer
  - Validation failures
    - invalid input, business rule violations, authorization failures, resource not found
- External/integration layer
  - Third-party API failures, timeouts, unexpected response formats
- Infrastructure
  - Server out of memory, unhandled exceptions, configuration errors

Classify errors by recoverability

- Client errors
  - Fix your input, log in again, you don't have permission — the user can take action.
- Server errors
  - Something broke internally — the user can only retry or wait.
- Transient errors
  - Network hiccup, database momentarily unavailable — retrying might work.

Consistent error shape

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Description of error",
    "details": [ ... per-field details ... ],
    "timestamp": "...",
    "traceId": "..."
  }
}
```

- **code** is machine-readable
- **message** is human-readable
- **details** enables per-field validation feedback
- **traceId** links to server-logs for debugging

Map error codes to HTTP status codes

- 400 → VALIDATION_ERROR, INVALID_INPUT
- 401 → AUTHENTICATION_REQUIRED
- 403 → AUTHORIZATION_FAILED
- 404 → RESOURCE_NOT_FOUND
- 409 → CONFLICT (e.g., duplicate entry)
- 422 → BUSINESS_RULE_VIOLATION
- 500 → INTERNAL_ERROR
- 503 → SERVICE_UNAVAILABLE

UNIVERSITY OF SZEGED
Department of Software Engineering
UNIVERSITAS SCIENTIARUM SZEGEDIENSIS

Error propagation across layers

- **Errors transform as they cross layer boundaries**
- A raw database constraint violation should never reach the user.
- Each layer catches, interprets, and re-throws errors in its own vocabulary.

Layer	Error
Database layer	duplicate key value violates unique constraint "categories_name_key"
Repository/data access layer	DuplicateResourceError("Category with this name already exists")
Service/business logic layer	May add context or just let it propagate
API/controller layer	HTTP 409 with the standard error shape
Frontend layer	Shows a localized, user-friendly message near the input field

Frontend error-handling strategy

- Global error state
  - e.g.: session expired, server unreachable
  - typically shown as a toast or banner
- Local error state
  - e.g.: form validation errors
  - shown inline next to the relevant field

Frontend error-handling flow

1. Make an API call
2. If the response is not OK, parse the error body
3. Based on error.code, decide the UX:
   a. AUTHENTICATION_REQUIRED → redirect to login
   b. VALIDATION_ERROR → map ***details*** to form fields, show inline
   c. INTERNAL_ERROR → show a generic “please try again” toast
   d. SERVICE_UNAVAILABLE → show retry option